

Code-Layout, Readability and Reusability

Date	15 March 2026
Team ID	[Your Team ID]
Project Name	Personalized Networking Assistant
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the Personalized Networking Assistant codebase in terms of its structure, readability, and reusability. The backend is organized into independent FastAPI service modules (event analysis, conversation generation, and fact-checking), while the Streamlit frontend is kept separate from business logic. Consistent naming conventions, docstrings, and modular design ensure the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes/No/Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	PEP 8 formatting maintained across all modules
2	Proper File Structure	Files and folders are logically organized	Yes	Separate directories for routes, services, models, and frontend
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	e.g., event_description, extracted_themes, fact_summary
4	Function / Method Names	Functions are descriptively named	Yes	e.g., extract_themes(), generate_starters(), verify_fact()
5	Code Comments	Inline and block comments explain logic	Yes	Docstrings added for all service and route functions
6	Modular Design	Code is split into reusable functions/modules	Yes	event_analyzer.py, topic_generator.py and fact_checker.py are independent modules
7	No Redundant Code	Duplicate or unused code is removed	Yes	Common utilities refactored into a shared helpers module
8	Error Handling	Exceptions and errors are handled gracefully	Yes	Try/except blocks with custom HTTP exceptions in all API routes

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High/Medium/Low)
1	event_analyzer.py	Python (Transformers - DistilBERT)	Theme extraction for both conversation generation and fact-check query building	High
2	topic_generator.py	Python (Transformers - GPT-2)	Conversation starter generation across all event categories	High
3	fact_checker.py	Python (Wikipedia API wrapper)	Fact verification requests from both the generation flow and the standalone fact-check feature	High
4	history_logger.py	Python (JSON persistence)	Logging conversation history and feedback across sessions	Medium
5	api_client.py (frontend helper)	Python (Streamlit + Requests)	Common wrapper used by all Streamlit UI sections to call FastAPI endpoints	Medium

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Clear separation of routes, services, and frontend layers
Readability	4	Descriptive naming and consistent formatting throughout
Reusability	5	Core NLP and fact-checking modules are fully decoupled and reusable
Documentation / Comments	4	Docstrings present; a few utility scripts could use more detail
Overall Score	4.5	Well-structured, maintainable, and extensible codebase